Syntax Analysis

1

```
while (ip < z)
    ++ip;
```

2

w	h	i	l	e		(	i	p		<		z	)	\n	\t	+	+	i	p	;

```
while (ip < z)
    ++ip;
```

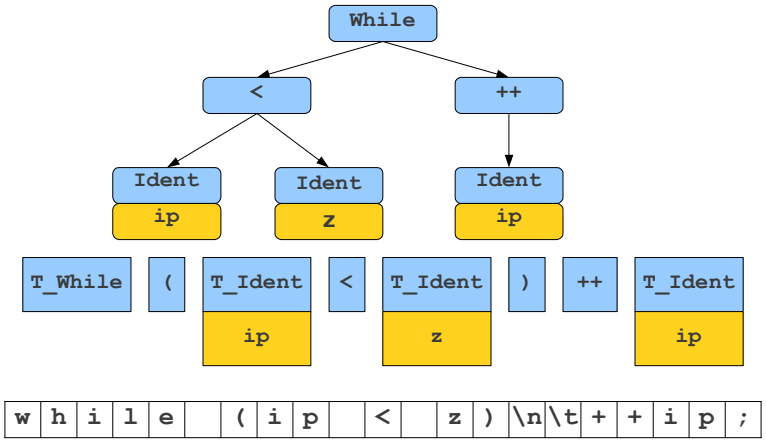
3

T_While	(	T_Ident	<	T_Ident	)	++	T_Ident
		ip		z			ip

w	h	i	l	e		(	i	p		<		z	)	\n	\t	+	+	i	p	;

```
while (ip < z)
    ++ip;
```

4

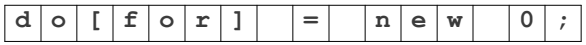



```
while (ip < z)
    ++ip;
```

5

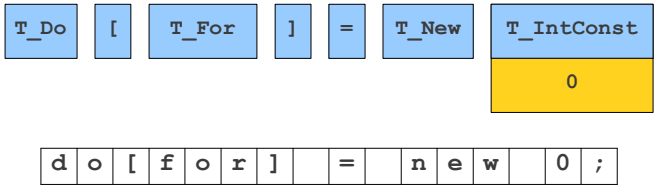
```
do[for] = new 0;
```

6




```
do[for] = new 0;
```

7




```
do[for] = new 0;
```

8

What is Syntax Analysis?

- After lexical analysis (scanning), we have a series of tokens.
- In **syntax analysis** (or **parsing**), we want to interpret what those tokens mean.
- Goal: Recover the *structure* described by that series of tokens.
- Goal: Report *errors* if those tokens do not properly encode a structure.

9

Outline

- Formalisms for syntax analysis.
  - Context-Free Grammars
  - Derivations
  - Concrete and Abstract Syntax Trees
  - Ambiguity
- Parsing algorithms.
  - Top-Down Parsing
  - Bottom-Up Parsing

10

Formal Languages

- An **alphabet** is a set Σ of symbols that act as letters.
- A **language** over Σ is a set of strings made from symbols in Σ.
- When scanning, our alphabet was ASCII or Unicode characters. We produced tokens.
- When parsing, our alphabet is the set of tokens produced by the scanner.

11

The Limits of Regular Languages

- When scanning, we used regular expressions to define each token.
- Unfortunately, regular expressions are (usually) too weak to define programming languages.
  - Cannot define a regular expression matching all expressions with properly balanced parentheses.
  - Cannot define a regular expression matching all functions with properly nested block structure.
- We need a more powerful formalism.

12

Context-Free Grammars

A CFG is a quadruple (N, T, R, S) where

- N is the set of non-terminal symbols
- T is the set of terminal symbols
- $S \in N$ is the starting symbol
- $R \subseteq N \times (N \cup T)^*$ is a set of rules

Example: The grammar of nested parentheses

G = (N, T, R, S) where

- N = {S}
- T ={ (,) }
- R ={ S→ (S) , S→SS, S→ε }

13

Context-Free Grammars

- A **context-free grammar** (or **CFG**) is a formalism for defining languages.
- Can define the **context-free languages**, a strict superset of the the regular languages.
- CFGs are best explained by example…

14

Arithmetic Expressions

- Suppose we want to describe all legal arithmetic expressions using addition, subtraction, multiplication, and division.
- Here is one possible CFG:

E → int
E → E Op E
E → (E)
Op → +
Op → -
Op → *
Op → /

E
⇒ E Op E
⇒ E Op (E)
⇒ E Op (E Op E)
⇒ E * (E Op E)
⇒ int * (E Op E)
⇒ int * (int Op E)
⇒ int * (int Op int)
⇒ int * (int + int)

15

Arithmetic Expressions

- Suppose we want to describe all legal arithmetic expressions using addition, subtraction, multiplication, and division.
- Here is one possible CFG:

E → int
E → E Op E
E → (E)
Op → +
Op → -
Op → *
Op → /

E
⇒ E Op E
⇒ E Op int
⇒ int Op int
⇒ int / int

16

Context-Free Grammars

- Formally, a context-free grammar is a collection of four objects:
  - A set of **nonterminal symbols** (or **variables**),
  - A set of **terminal symbols**,
  - A set of **production rules** saying how each nonterminal can be converted by a string of terminals and nonterminals, and
  - A **start symbol** that begins the derivation.

E → int
E → E Op E
E → (E)
Op → +
Op → -
Op → *
Op → /

17

A Notational Shorthand

E → int
E → E Op E
E → (E)
Op → +
Op → -
Op → *
Op → /

18

A Notational Shorthand

E → int | E Op E | (E)
Op → + | - | * | /

19

Not Notational Shorthand

- The syntax for regular expressions does not carry over to CFGs.
- Cannot use *, |, or parentheses.

S → a*b

20

Not Notational Shorthand

- The syntax for regular expressions does not carry over to CFGs.
- Cannot use *, |, or parentheses.

$S \rightarrow Ab$

21

Not Notational Shorthand

- The syntax for regular expressions does not carry over to CFGs.
- Cannot use *, |, or parentheses.

$S \rightarrow Ab$
$A \rightarrow Aa \mid \varepsilon$

22

Not Notational Shorthand

- The syntax for regular expressions does not carry over to CFGs.
- Cannot use *, |, or parentheses.

$S \rightarrow a(b|c^*)$

23

Not Notational Shorthand

- The syntax for regular expressions does not carry over to CFGs.
- Cannot use *, |, or parentheses.

$S \rightarrow aX$
$X \rightarrow (b|c^*)$

24

Not Notational Shorthand

- The syntax for regular expressions does not carry over to CFGs.
- Cannot use *, |, or parentheses.

S →aX
X →b| c*

25

Not Notational Shorthand

- The syntax for regular expressions does not carry over to CFGs.
- Cannot use *, |, or parentheses.

S →aX
X →b| C

26

Not Notational Shorthand

- The syntax for regular expressions does not carry over to CFGs.
- Cannot use *, |, or parentheses.

S →aX
X →b | C
C →Cc | ε

27

More Context-Free Grammars

- Chemicals!

$C_{19}H_{14}O_5S$

$Cu_3(CO_3)_2(OH)_2$

MnO_4^-

S^{2-}

Form →Cmp |Cmp Ion
Cmp →Term / Term Num / Cmp Cmp
Term →Elem |(Cmp)
Elem →H|He|Li|Be|B|C| …
Ion →+|-|IonNum +|IonNum -
IonNum →2|3|4|…
Num →1|IonNum

28

CFGs for Chemistry

Form → Cmp | Cmp Ion
Cmp → Term / Term Num / Cmp Cmp
Term → Elem | (Cmp)
Elem → H | He | Li | Be | B | C | …
Ion → + | - | IonNum + | IonNum -
IonNum → 2 | 3 | 4 | …
Num → 1 | IonNum

Form
⇒ Cmp Ion
⇒ Cmp Cmp Ion
⇒ Cmp Term Num Ion
⇒ Term Term Num Ion
⇒ Elem Term Num Ion
⇒ Mn Term Num Ion
⇒ Mn Elem Num Ion
⇒ MnO Num Ion
⇒ MnO IonNum Ion
⇒ MnO_4 Ion
⇒ MnO_4^-

29

CFGs for Programming Languages

```
BLOCK   → STMT
        | {STMTS }

STMTS   → ε
        | STMT STMTS

STMT    → EXPR;
        | if (EXPR) BLOCK
        | while (EXPR) BLOCK
        | do BLOCK while (EXPR);
        | BLOCK
        | …

EXPR    → identifier
        | constant
        | EXPR + EXPR
        | EXPR – EXPR
        | EXPR * EXPR
        | …
```

30

Statement

STATEMENT-> IF_ STATEMENT| FOR_STATEMENT| WHILE_STATEMENT|……|;

31

If statement

IF_STATEMENT->if (COND_EXP) then STATEMENT | | if (COND_EXP) then STATEMENT else STATEMENT

```
IF(A<B)
  IF(C<d)
   ;
ELSE
   IF(
```

32

S->A
A->B|(S)

33

Some CFG Notation

- We will be discussing generic transformations and operations on CFGs Let's standardize our notation.
-

34

Some CFG Notation

- Capital letters at the beginning of the alphabet will represent nonterminals.
  - i.e. **A**, **B**, **C**, **D**
- Lowercase letters at the end of the alphabet will represent terminals.
  - i.e. **t**, **u**, **v**, **w**
- Lowercase Greek letters will represent arbitrary strings of terminals and nonterminals.
  - i.e. α, γ, ω

35

Examples

- We might write an arbitrary production as

$$\mathbf{A} \rightarrow \omega$$

- We might write a string of a nonterminal followed by a terminal as

$$\mathbf{A}\mathbf{t}$$

- We might write an arbitrary production containing a nonterminal followed by a terminal as

$$\mathbf{B} \rightarrow \alpha\mathbf{A}\mathbf{t}\omega$$

36

Derivations

```
  E
⇒ E Op E
⇒ E Op (E)
⇒ E Op (E Op E)
⇒ E * (E Op E)
⇒ int * (E Op E)
⇒ int * (int Op E)
⇒ int * (int Op int)
⇒ int * (int + int)
```

- This sequence of steps is called a **derivation**.
- A string $\alpha A \omega$ **yields** string $\alpha \gamma \omega$ iff $A \rightarrow \gamma$ is a production.
- If α yields β, we write $\alpha \Rightarrow \beta$.
- We say that α **derives** β iff there is a sequence of strings where

$$\alpha \Rightarrow \alpha_1 \Rightarrow \alpha_2 \Rightarrow \ldots \Rightarrow \beta$$

- If α derives β, we write $\alpha \Rightarrow^* \beta$.

Leftmost Derivations

```
BLOCK → STMT
      | {STMTS }

STMTS → ε
      | STMT STMTS

STMT  → EXPR;
      | if (EXPR) BLOCK
      | while (EXPR) BLOCK
      | do BLOCK while (EXPR);
      | BLOCK
      | …

EXPR  → identifier
      | constant
      | EXPR +EXPR
      | EXPR -EXPR
      | EXPR *EXPR
      | EXPR =EXPR
      | …
```

```
  STMTS
⇒ STMT STMTS
⇒ EXPR; STMTS
⇒ EXPR = EXPR; STMTS
⇒ id = EXPR; STMTS
⇒ id = EXPR + EXPR; STMTS
⇒ id = id + EXPR; STMTS
⇒ id = id + constant; STMTS
⇒ id = id + constant;
```


a bc x 9)
a==
for+-==[while!1-[35=23
===
89abcf945(

Leftmost Derivations

- A **leftmost derivation** is a derivation in which each step expands the leftmost nonterminal.
- A **rightmost derivation** is a derivation in which each step expands the rightmost nonterminal.
- These will be of great importance when we talk about parsing next week.

Related Derivations

E	E
⇒ E Op E	⇒ E Op E
⇒ int Op E	⇒ E Op (E)
⇒ int * E	⇒ E Op (E Op E)
⇒ int * (E)	⇒ E Op (E Op int)
⇒ int * (E Op E)	⇒ E Op (E + int)
⇒ int * (int Op E)	⇒ E Op (int + int)
⇒ int * (int + E)	⇒ E * (int + int)
⇒ int * (int + int)	⇒ int * (int + int)

41

Derivations Revisited

- A derivation encodes two pieces of information:
  - What productions were applied produce the resulting string from the start symbol?
  - In what order were they applied?
- Multiple derivations might use the same productions, but apply them in a different order.

42

Parse Trees

E

43

Parse Trees

E

E

44

Parse Trees

E

E
⇒ E Op E

45

Parse Trees

E

E
⇒ E Op E

46

Parse Trees

E

E
⇒ E Op E
⇒ int Op E

47

Parse Trees

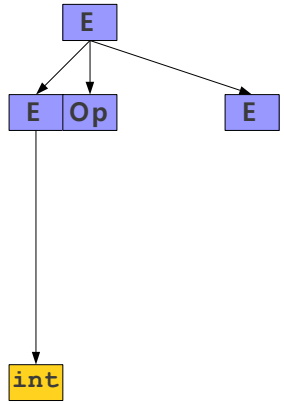
E

E
⇒ E Op E
⇒ int Op E

48

Parse Trees

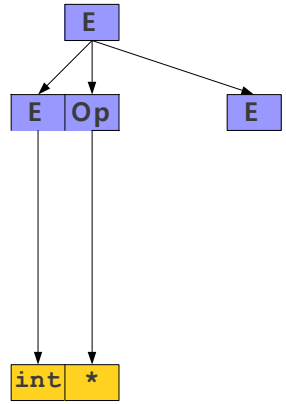
E
⇒ E Op E
⇒ int Op E
⇒ int * E



49

Parse Trees

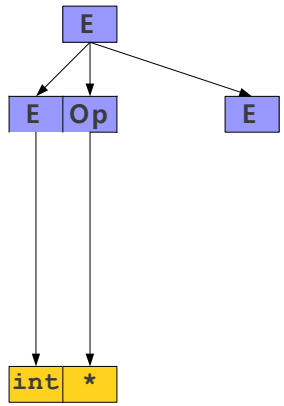
E
⇒ E Op E
⇒ int Op E
⇒ int * E



50

Parse Trees

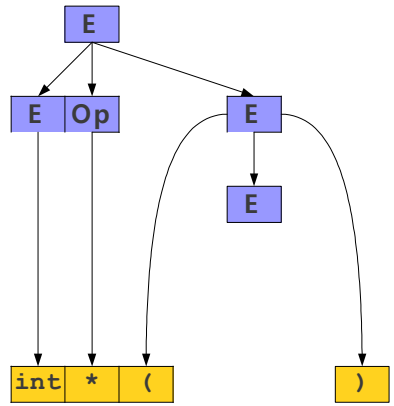
E
⇒ E Op E
⇒ int Op E
⇒ int * E
⇒ int * (E)



51

Parse Trees

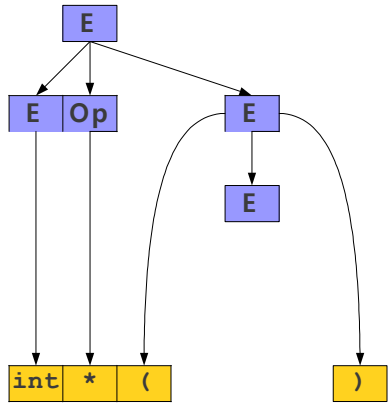
E
⇒ E Op E
⇒ int Op E
⇒ int * E
⇒ int * (E)



52

Parse Trees

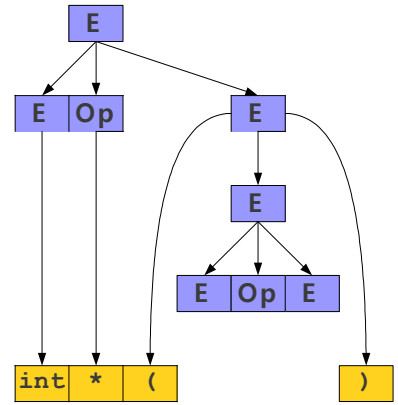
E
⇒ E Op E
⇒ int Op E
⇒ int * E
⇒ int * (E)
⇒ int * (E Op E)



53

Parse Trees

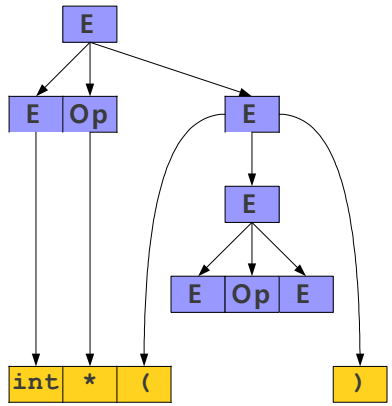
E
⇒ E Op E
⇒ int Op E
⇒ int * E
⇒ int * (E)
⇒ int * (E Op E)



54

Parse Trees

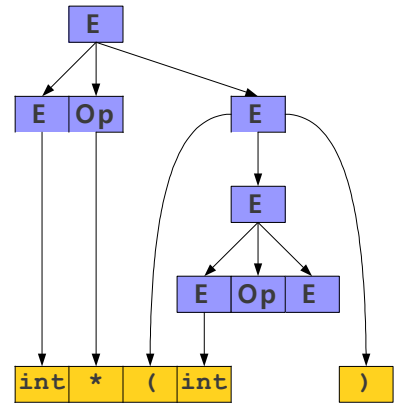
E
⇒ E Op E
⇒ int Op E
⇒ int * E
⇒ int * (E)
⇒ int * (E Op E)
⇒ int * (int Op E)



55

Parse Trees

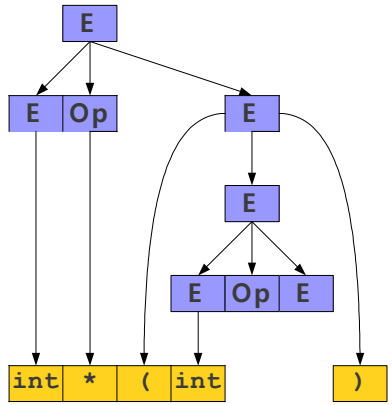
E
⇒ E Op E
⇒ int Op E
⇒ int * E
⇒ int * (E)
⇒ int * (E Op E)
⇒ int * (int Op E)



56

Parse Trees

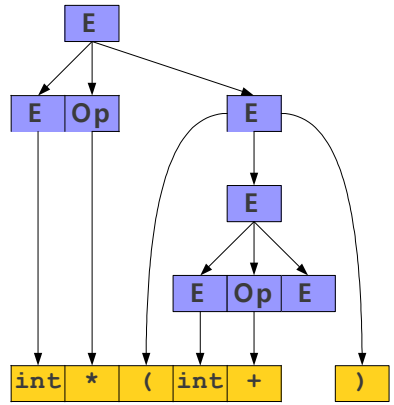
E
⇒ E Op E
⇒ int Op E
⇒ int * E
⇒ int * (E)
⇒ int * (E Op E)
⇒ int * (int Op E)
⇒ int * (int + E)



57

Parse Trees

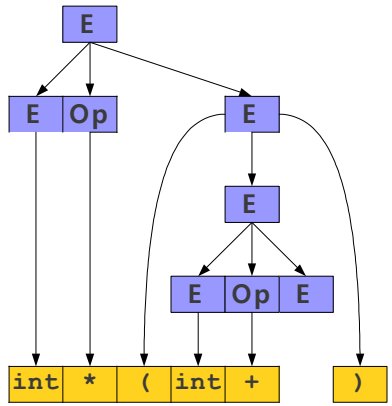
E
⇒ E Op E
⇒ int Op E
⇒ int * E
⇒ int * (E)
⇒ int * (E Op E)
⇒ int * (int Op E)
⇒ int * (int + E)



58

Parse Trees

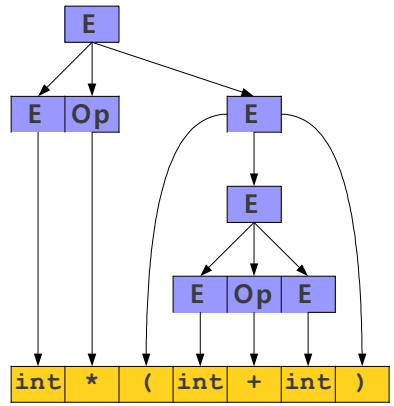
E
⇒ E Op E
⇒ int Op E
⇒ int * E
⇒ int * (E)
⇒ int * (E Op E)
⇒ int * (int Op E)
⇒ int * (int + E)
⇒ int * (int + int)



59

Parse Trees

E
⇒ E Op E
⇒ int Op E
⇒ int * E
⇒ int * (E)
⇒ int * (E Op E)
⇒ int * (int Op E)
⇒ int * (int + E)
⇒ int * (int + int)



60

Parse Trees

E

61

Parse Trees

E

E

62

Parse Trees

E

E

⇒ E Op E

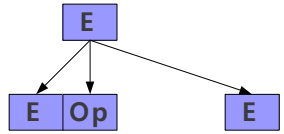
63

Parse Trees

E

E

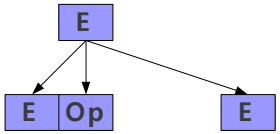
⇒ E Op E



64

Parse Trees

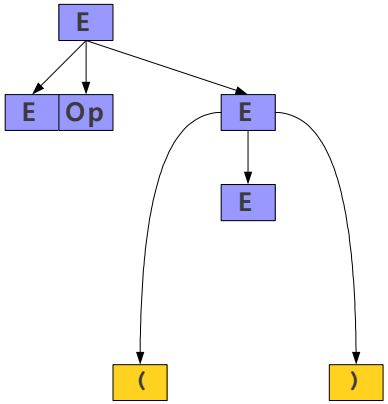
E
⇒ E Op E
⇒ E Op (E)



65

Parse Trees

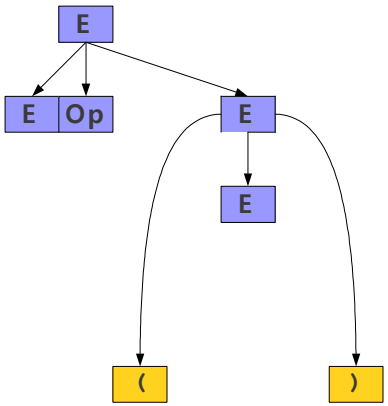
E
⇒ E Op E
⇒ E Op (E)



66

Parse Trees

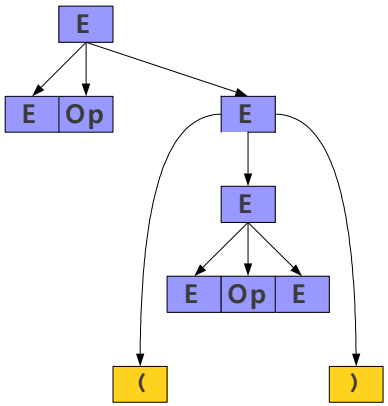
E
⇒ E Op E
⇒ E Op (E)
⇒ E Op (E Op E)



67

Parse Trees

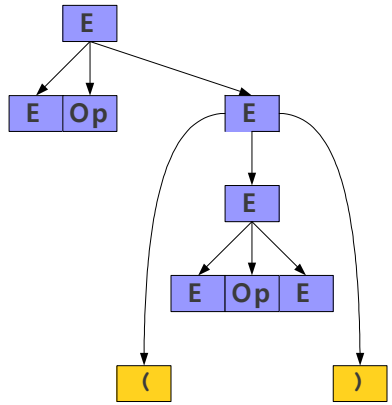
E
⇒ E Op E
⇒ E Op (E)
⇒ E Op (E Op E)



68

Parse Trees

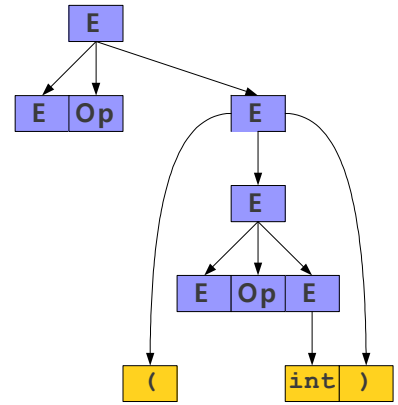
E
⇒ E Op E
⇒ E Op (E)
⇒ E Op (E Op E)
⇒ E Op (E Op int)



69

Parse Trees

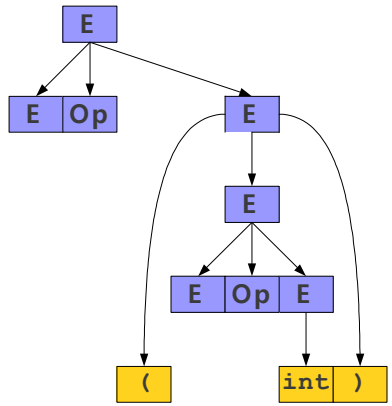
E
⇒ E Op E
⇒ E Op (E)
⇒ E Op (E Op E)
⇒ E Op (E Op int)



70

Parse Trees

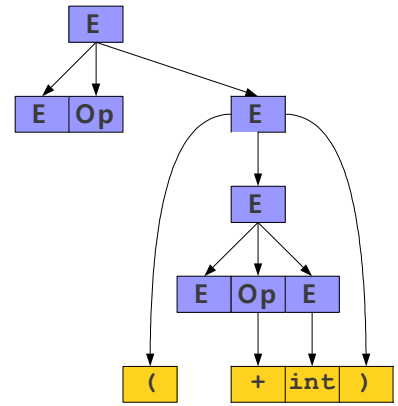
E
⇒ E Op E
⇒ E Op (E)
⇒ E Op (E Op E)
⇒ E Op (E Op int)
⇒ E Op (E + int)



71

Parse Trees

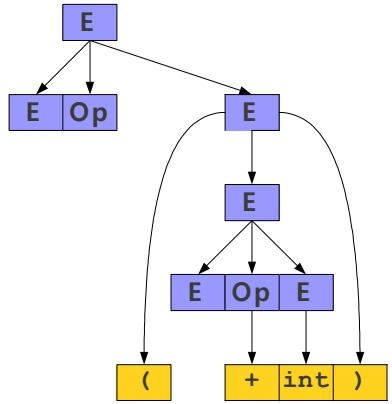
E
⇒ E Op E
⇒ E Op (E)
⇒ E Op (E Op E)
⇒ E Op (E Op int)
⇒ E Op (E + int)



72

Parse Trees

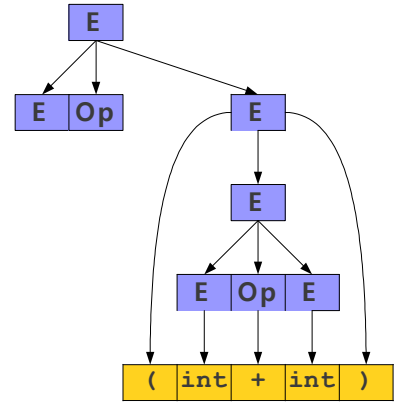
E
⇒ E Op E
⇒ E Op (E)
⇒ E Op (E Op E)
⇒ E Op (E Op int)
⇒ E Op (E + int)
⇒ E Op (int + int)



73

Parse Trees

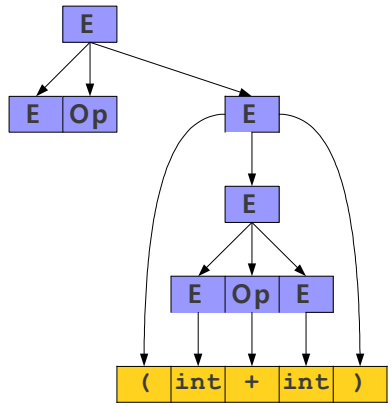
E
⇒ E Op E
⇒ E Op (E)
⇒ E Op (E Op E)
⇒ E Op (E Op int)
⇒ E Op (E + int)
⇒ E Op (int + int)



74

Parse Trees

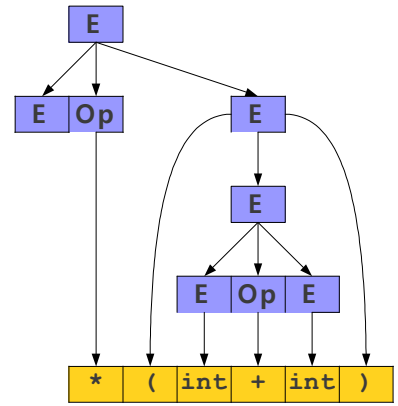
E
⇒ E Op E
⇒ E Op (E)
⇒ E Op (E Op E)
⇒ E Op (E Op int)
⇒ E Op (E + int)
⇒ E Op (int + int)
⇒ E * (int + int)



75

Parse Trees

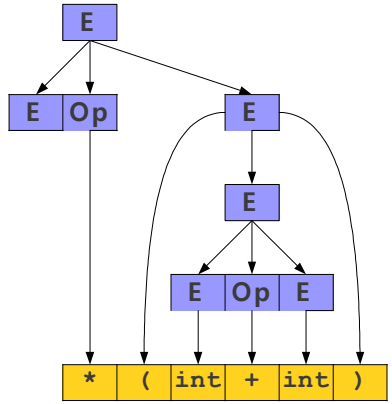
E
⇒ E Op E
⇒ E Op (E)
⇒ E Op (E Op E)
⇒ E Op (E Op int)
⇒ E Op (E + int)
⇒ E Op (int + int)
⇒ E * (int + int)



76

Parse Trees

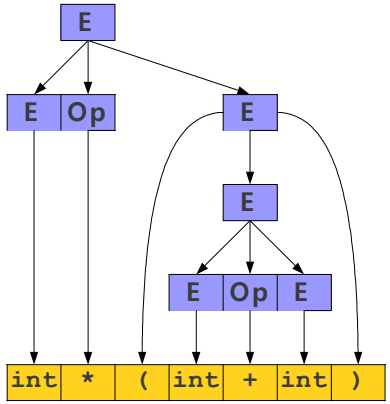
E
⇒ E Op E
⇒ E Op (E)
⇒ E Op (E Op E)
⇒ E Op (E Op int)
⇒ E Op (E + int)
⇒ E Op (int + int)
⇒ E * (int + int)
⇒ int * (int + int)



77

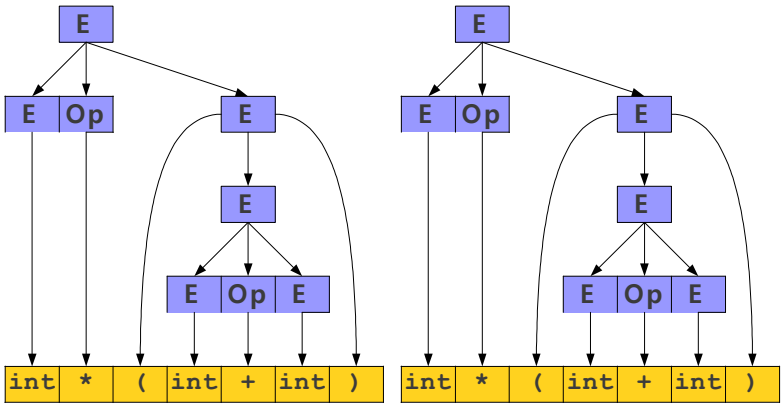
Parse Trees

E
⇒ E Op E
⇒ E Op (E)
⇒ E Op (E Op E)
⇒ E Op (E Op int)
⇒ E Op (E + int)
⇒ E Op (int + int)
⇒ E * (int + int)
⇒ int * (int + int)



78

For Comparison



79

Parse Trees

- A **parse tree** is a tree encoding the steps in a derivation.
- Internal nodes represent nonterminal symbols used in the production.
- Inorder walk of the leaves contains the generated string.
- Encodes what productions are used, not the order in which those productions are applied.

80

The Goal of Parsing

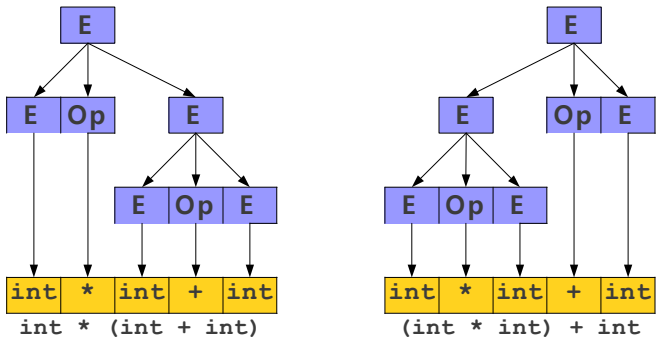
- Goal of syntax analysis: Recover the **structure** described by a series of tokens.
- If language is described as a CFG, goal is to recover a parse tree for the the input string.
  - Usually we do some simplifications on the tree; more on that later.
-

81

Challenges in Parsing

82

A Serious Problem



83

Ambiguity

- A CFG is said to be **ambiguous** if there is at least one string with two or more parse trees.
- Note that ambiguity is a property of *grammars*, not *languages*.
- There is no algorithm for converting an arbitrary ambiguous grammar into an unambiguous one.
  - Some languages are inherently ambiguous, meaning that no unambiguous grammar exists for them.
- There is no algorithm for detecting whether an arbitrary grammar is ambiguous.

84

Is Ambiguity a Problem?

- Depends on **semantics**.

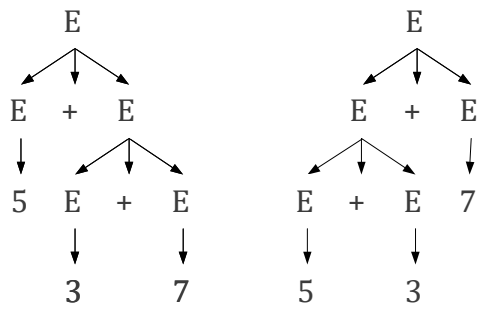
E → **int** | **E + E**

85

Is Ambiguity a Problem?

- Depends on **semantics**.

E → **int** | **E + E**



86

Is Ambiguity a Problem?

- Depends on **semantics**.

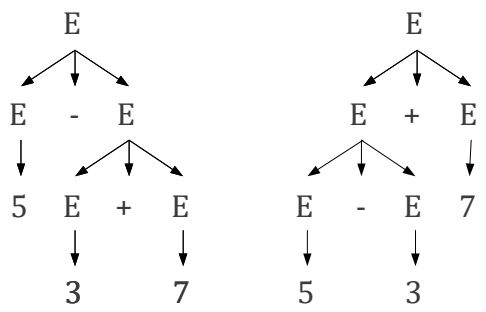
E → **int** | **E + E** | **E – E**

87

Is Ambiguity a Problem?

- Depends on **semantics**.

E → **int** | **E + E** | **E – E**



88

Resolving Ambiguity

- If a grammar can be made unambiguous at all, it is usually made unambiguous through **layering**.
- Have exactly one way to build each piece of the string.
- Have exactly one way of combining those pieces back together.

89

Example: Balanced Parentheses

- Consider the language of all strings of balanced parentheses.
- Examples:
  - ε
  - ()
  - (()())
  - ((())))(())()
- Here is one possible grammar for balanced parentheses:

$$P \rightarrow \varepsilon \mid PP \mid (P)$$

90

Balanced Parentheses

- Given the grammar $P \rightarrow \varepsilon \mid PP \mid (P)$
- How might we generate the string (()())?

91

Balanced Parentheses

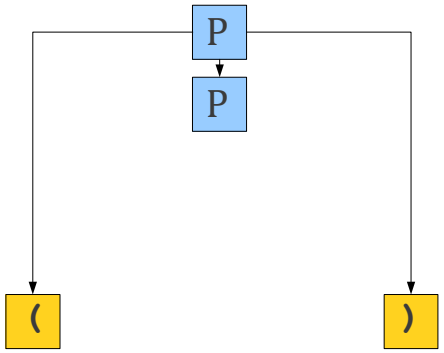
- Given the grammar $P \rightarrow \varepsilon \mid PP \mid (P)$
- How might we generate the string (()())?

P

92

Balanced Parentheses

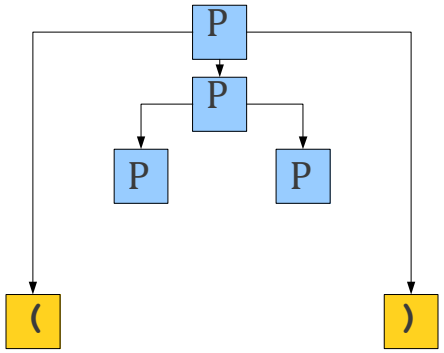
- Given the grammar **P** → **ε** | **PP** | **(P)**
- How might we generate the string **(()())**?



93

Balanced Parentheses

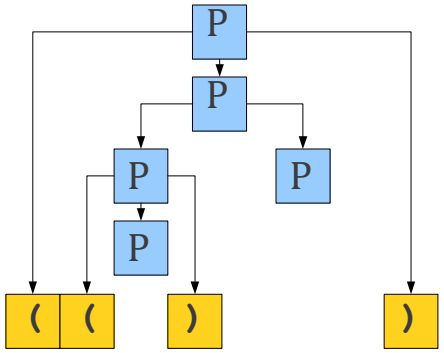
- Given the grammar **P** → **ε** | **PP** | **(P)**
- How might we generate the string **(()())**?



94

Balanced Parentheses

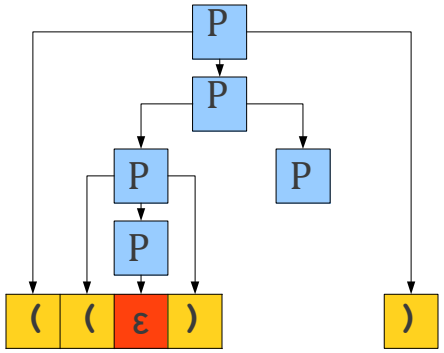
- Given the grammar **P** → **ε** | **PP** | **(P)**
- How might we generate the string **(()())**?



95

Balanced Parentheses

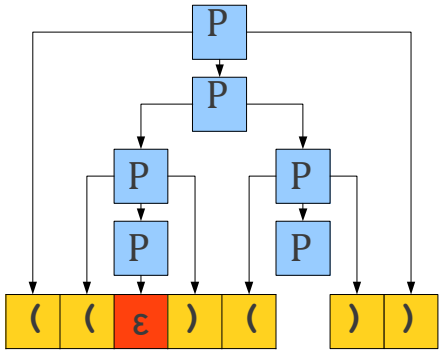
- Given the grammar **P** → **ε** | **PP** | **(P)**
- How might we generate the string **(()())**?



96

Balanced Parentheses

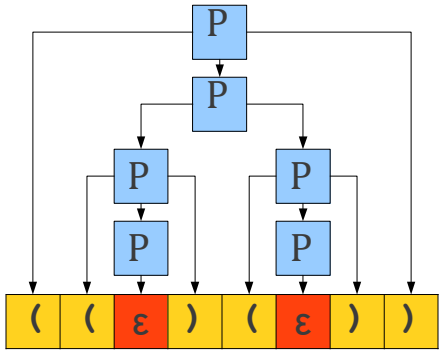
- Given the grammar **P** → **ε** | **PP** | **(P)**
- How might we generate the string **(()())**?



97

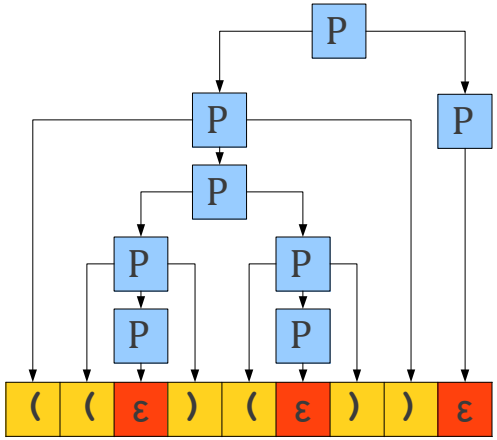
Balanced Parentheses

- Given the grammar **P** → **ε** | **PP** | **(P)**
- How might we generate the string **(()())**?



98

Balanced Parentheses



99

How to resolve this ambiguity?

100

(	(	)	(	)	)	(	)	(	(	)	)

101

(	(	)	(	)	)	(	)	(	(	)	)

102

(	(	)	(	)	)	(	)	(	(	)	)

103

(	(	)	(	)	)	(	)	(	(	)	)

104

Rethinking Parentheses

- A string of balanced parentheses is a sequence of strings that are themselves balanced parentheses.
- To avoid ambiguity, we can build the string in two steps:
  - Decide how many different substrings we will glue together.
  - Build each substring independently.

Building Parentheses

- Spread a string of parentheses across the string. There is exactly one way to do this for any number of parentheses.
- Expand out each substring by adding in parentheses and repeating.

$$S \rightarrow P\ S \mid \varepsilon$$
$$P \rightarrow (\ S\ )$$

Building Parentheses

$$S \rightarrow P\ S \mid \varepsilon$$
$$P \rightarrow (\ S\ )$$

S
⇒ PS
⇒ PPS
⇒ PP
⇒ (S) P
⇒ (S) (S)
⇒ (PS) (S)
⇒ (P) (S)
⇒ ((S)) (S)
⇒ (()) (S)
⇒ (()) ()

Context-Free Grammars

- A regular expression can be
  - Any letter
  - ε
  - The concatenation of regular expressions.
  - The union of regular expressions.
  - The Kleene closure of a regular expression.
  - A parenthesized regular expression.

Context-Free Grammars

- This gives us the following CFG:

  R → a | b | c | …

  R → "ε"

  R → RR

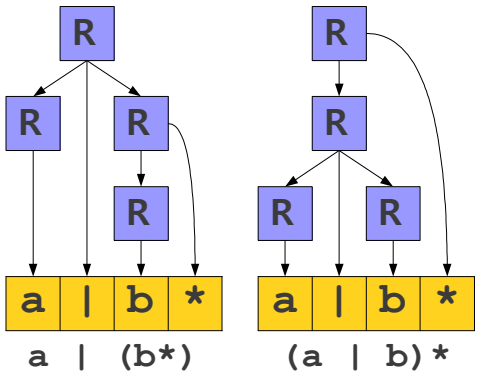
  R → R "|" R

  R → R*

  R → (R)

An Ambiguous Grammar

R → a | b | c | …
R → "ε"
R → RR
R → R "|" R
R → R*
R → (R)



Resolving Ambiguity

- We can try to resolve the ambiguity via layering:

  R → a | b | c | …

  R → "ε"

  R → RR

  R → R "|" R

  R → R*

  R → (R)

a	a	|	b	*

Resolving Ambiguity

- We can try to resolve the ambiguity via layering:

  R → a | b | c | …

  R → "ε"

  R → RR

  R → R "|" R

  R → R*

  R → (R)

a	a	|	b	*

Resolving Ambiguity

- We can try to resolve the ambiguity via layering:

  R → a | b | c | …

  R → "ε"

  R → RR

  R → R "|" R

  R → R*

  R → (R)

a	a	|	b	*

113

Resolving Ambiguity

- We can try to resolve the ambiguity via layering:

  R → a | b | c | …

  R → "ε"

  R → RR

  R → R "|" R

  R → R*

  R → (R)

a	a	|	b	*

114

Resolving Ambiguity

- We can try to resolve the ambiguity via layering:

  R → a | b | c | …

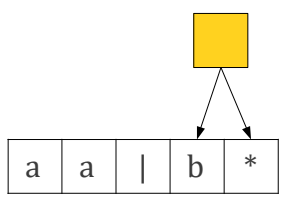
  R → "ε"

  R → RR

  R → R "|" R

  R → R*

  R → (R)



115

Resolving Ambiguity

- We can try to resolve the ambiguity via layering:

  R → a | b | c | …

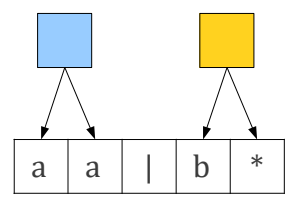
  R → "ε"

  R → RR

  R → R "|" R

  R → R*

  R → (R)

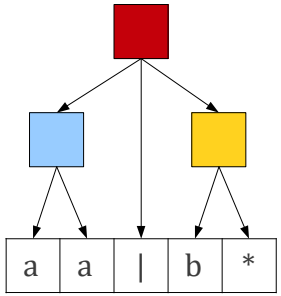


116

Resolving Ambiguity

- We can try to resolve the ambiguity via layering:

R → a | b | c | …
R → "ε"
R → RR
R → R "|" R
R → R*
R → (R)



Resolving Ambiguity

- We can try to resolve the ambiguity via layering:

R → a | b | c | …
R → "ε"
R → RR
R → R "|" R
R → R*
R → (R)

R → S | R "|" S
S → T | ST
T → U | T*
U → a | b | c | …
U → "ε"
U → (R)

Why is this unambiguous?

R → S | R "|" S
S → T | ST
T → U | T*
U → a | b | …
U → "ε"
U → (R)

Why is this unambiguous?

R → S | R "|" S
S → T | ST
T → U | T*
U → a | b | …
U → "ε"
U → (R)

Only generates "atomic" expressions

Why is this unambiguous?

R →S | R "|"S
S →T |ST
T →U |T*
U →a |b| …
U →"ε"
U →(R)

Puts stars onto atomic expressions

Only generates "atomic" expressions

Why is this unambiguous?

R →S | R "|"S
S →T |ST
T →U | T*
U →a |b| …
U →"ε"
U →(R)

Concatenates starred expressions

Puts stars onto atomic expressions

Only generates "atomic" expressions

Why is this unambiguous?

R →S | R "|"S
S →T |ST
T →U | T*
U →a |b| …
U →"ε"
U →(R)

Unions concatenated expressions

Concatenates starred expressions

Puts stars onto atomic expressions

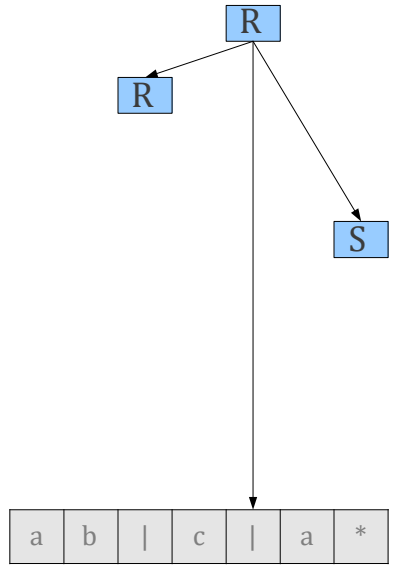
Only generates "atomic" expressions

R

R →S | R "|"S
S →T |ST
T →U |T*
U →a | b | c | …
U →"ε"
U →(R)

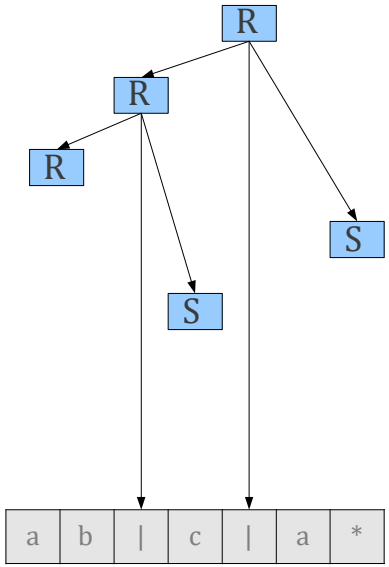
a	b	|	c	|	a	*

R →S | R "|"S
S →T |ST
T →U |T*
U → a | b | c | …
U →"ε"
U → (R)



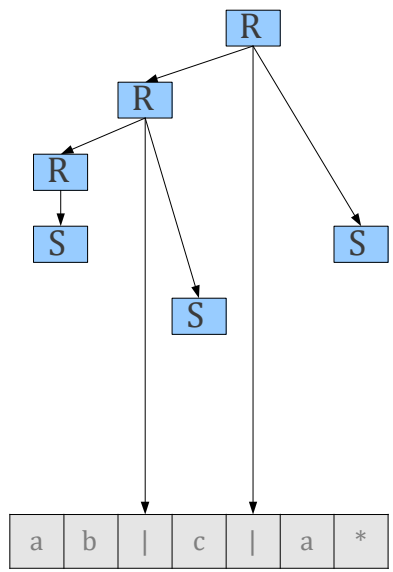
125

R →S | R "|"S
S →T |ST
T →U |T*
U → a | b | c | …
U →"ε"
U → (R)



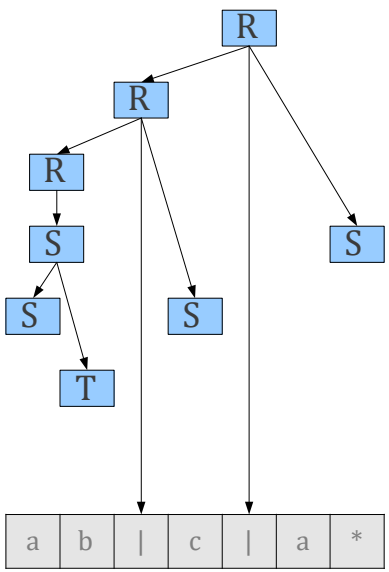
126

R →S | R "|"S
S →T |ST
T →U |T*
U → a | b | c | …
U →"ε"
U → (R)



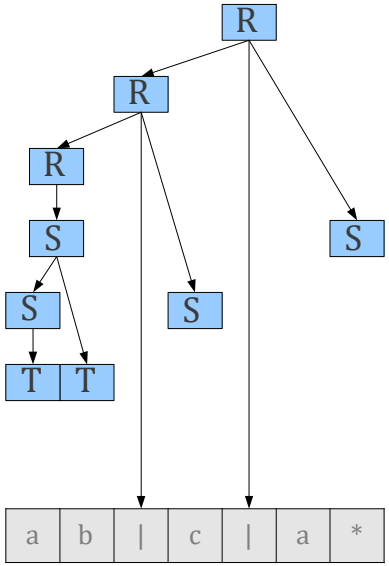
127

R →S | R "|"S
S →T |ST
T →U |T*
U → a | b | c | …
U →"ε"
U → (R)



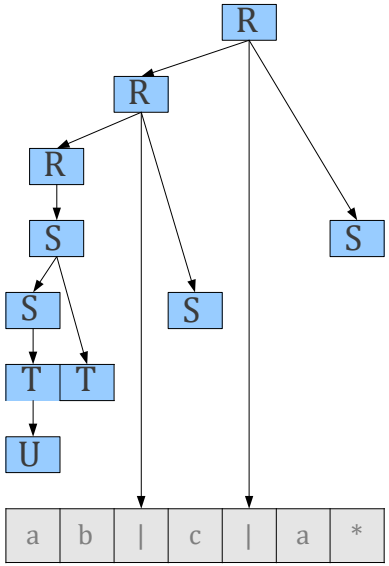
128

R →S | R "|"S
S →T |ST
T →U |T*
U → a | b | c | …
U →"ε"
U → (R)



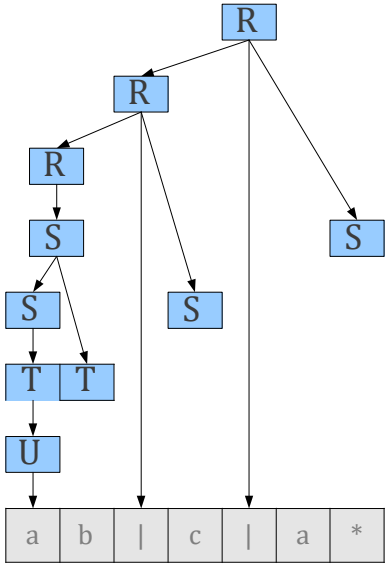
129

R →S | R "|"S
S →T |ST
T →U |T*
U → a | b | c | …
U →"ε"
U → (R)



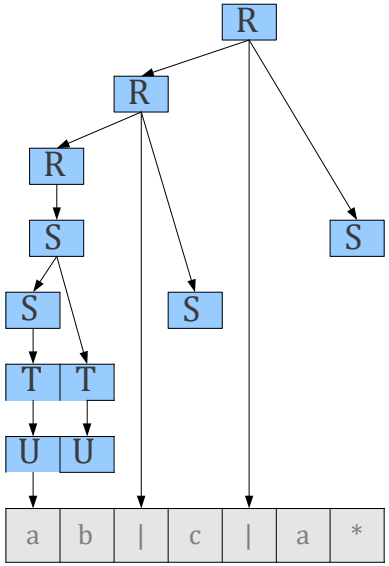
130

R →S | R "|"S
S →T |ST
T →U |T*
U → a | b | c | …
U →"ε"
U → (R)



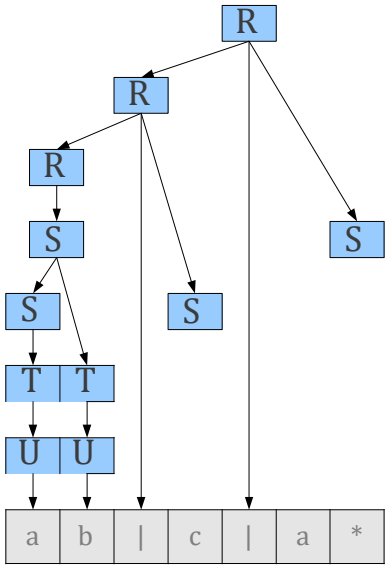
131

R →S | R "|"S
S →T |ST
T →U |T*
U → a | b | c | …
U →"ε"
U → (R)



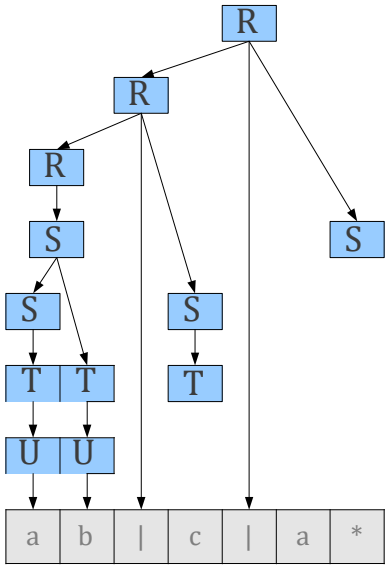
132

R →S | R "|"S
S →T |ST
T →U |T*
U → a | b | c | …
U →"ε"
U → (R)



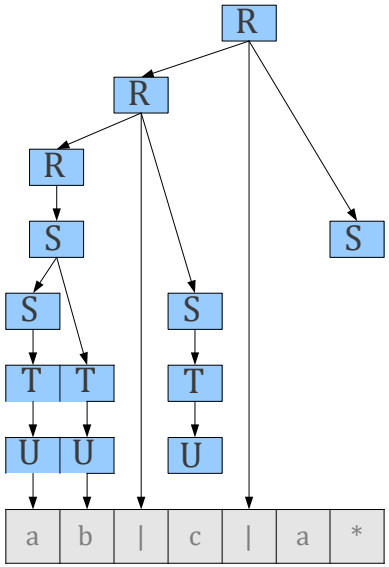
133

R →S | R "|"S
S →T |ST
T →U |T*
U → a | b | c | …
U →"ε"
U → (R)



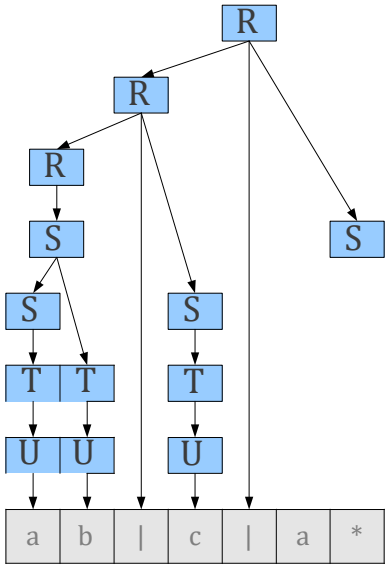
134

R →S | R "|"S
S →T |ST
T →U |T*
U → a | b | c | …
U →"ε"
U → (R)



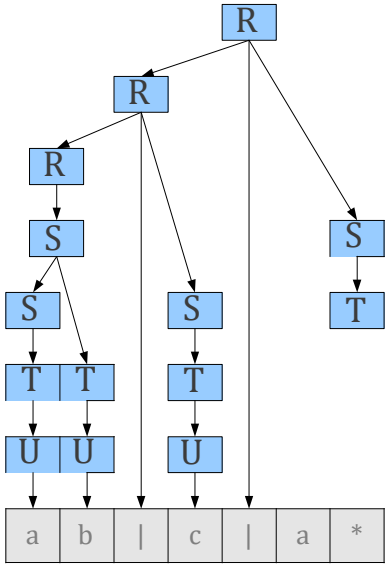
135

R →S | R "|"S
S →T |ST
T →U |T*
U → a | b | c | …
U →"ε"
U → (R)



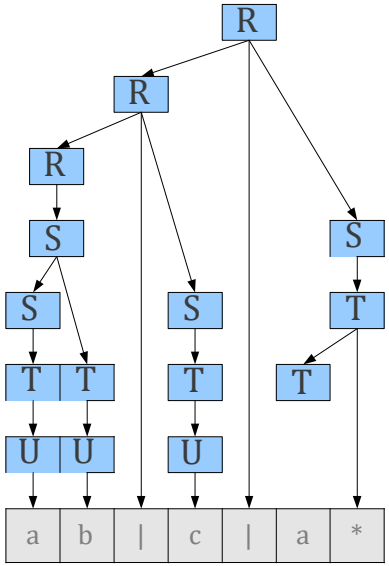
136

R →S | R "|"S
S →T |ST
T →U |T*
U → a | b | c | ...
U →"ε"
U → (R)



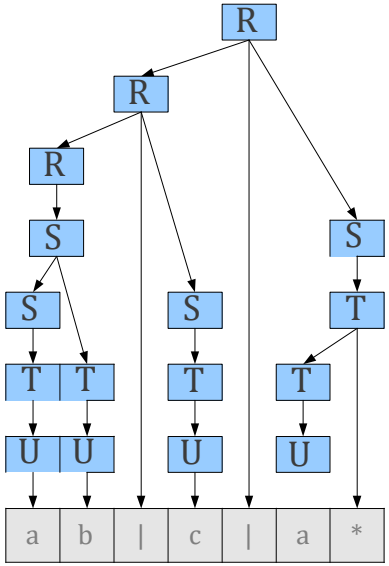
137

R →S | R "|"S
S →T |ST
T →U |T*
U → a | b | c | ...
U →"ε"
U → (R)



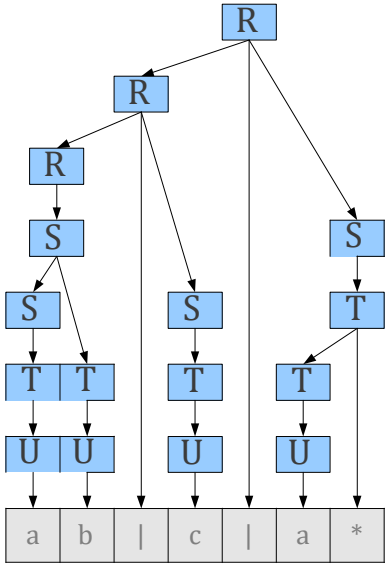
138

R →S | R "|"S
S →T |ST
T →U |T*
U → a | b | c | ...
U →"ε"
U → (R)



139

R →S | R "|"S
S →T |ST
T →U |T*
U → a | b | c | ...
U →"ε"
U → (R)



140

Precedence Declarations

- If we leave the world of pure CFGs, we can often resolve ambiguities through **precedence declarations**.
  - e.g. multiplication has higher precedence than addition, but lower precedence than exponentiation.
- Allows for unambiguous parsing of ambiguous grammars.
- We'll see how this is implemented later on.

The Structure of a Parse Tree

R →S | R "|"S
S →T |ST
T →U | T*
U →a |b| …
U →"ε"
U → (R)

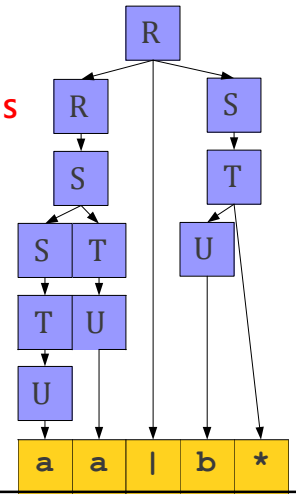
The Structure of a Parse Tree

R →S | R "|"S
S →T |ST
T →U | T*
U →a |b| …
U →"ε"
U → (R)

a	a	|	b	*

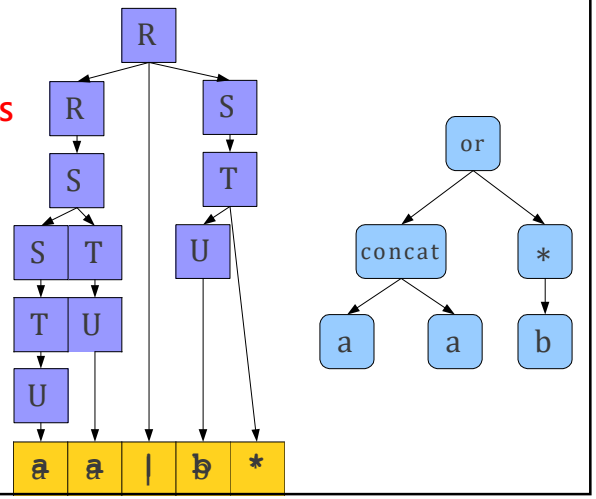
The Structure of a Parse Tree

R →S | R "|"S
S →T |ST
T →U |T*
U →a |b| …
U →"ε"
U → (R)



The Structure of a Parse Tree

R →S | R "|"S
S →T |ST
T →U |T*
U →a |b| …
U →"ε"
U →(R)

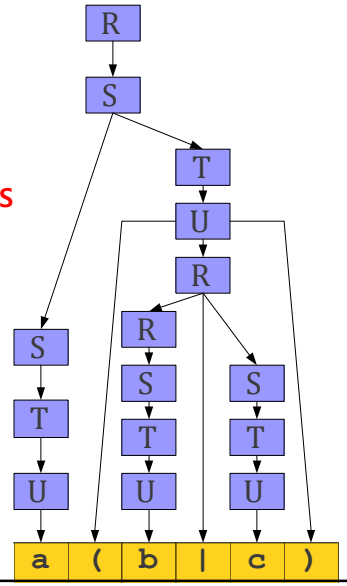


145

R →S | R "|"S
S →T |ST
T →U | T*
U →a |b| …
U →"ε"
U →(R)

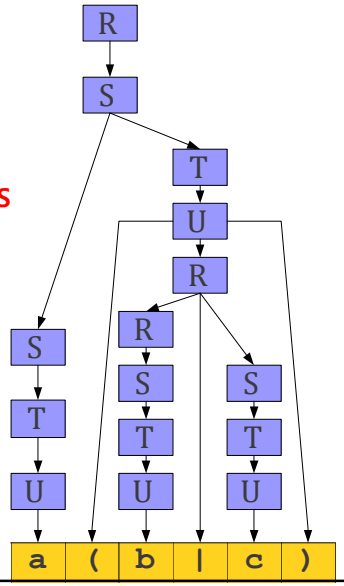
146

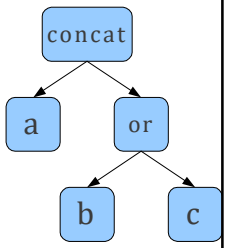
R →S | R "|"S
S →T |ST
T →U |T*
U →a |b| …
U →"ε"
U →(R)



147

R →S | R "|"S
S →T |ST
T →U |T*
U →a |b| …
U →"ε"
U →(R)





148

Abstract Syntax Trees (ASTs)

- A parse tree is a **concrete syntax tree**; it shows exactly how the text was derived.
- A more useful structure is an **abstract syntax tree**, which retains only the essential structure of the input.

How to build an AST?

- Typically done through **semantic actions**.
- Associate a piece of code to execute with each production.
- As the input is parsed, execute this code to build the AST.
  - Exact order of code execution depends on the parsing method used.
- This is called a **syntax-directed translation**.

Simple Semantic Actions

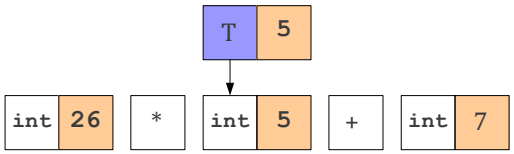
E → T + E	E_1.val = T.val + E_2.val
E → T	E.val = T.val
T → int	T.val = int.val
T → int * T	T.val = int.val * T.val
T → (E)	T.val = E.val

Simple Semantic Actions

E → T + E	E_1.val = T.val + E_2.val
E → T	E.val = T.val
T → int	T.val = int.val
T → int * T	T.val = int.val * T.val
T → (E)	T.val = E.val

Simple Semantic Actions

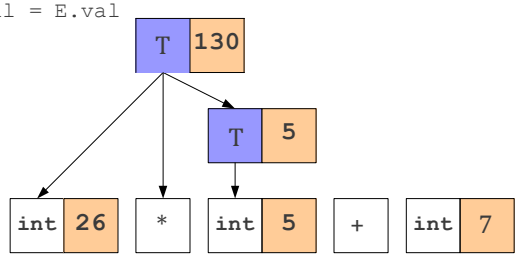
E → T + E	E_1.val = T.val + E_2.val
E → T	E.val = T.val
T → int	T.val = int.val
T → int * T	T.val = int.val * T.val
T → (E)	T.val = E.val



153

Simple Semantic Actions

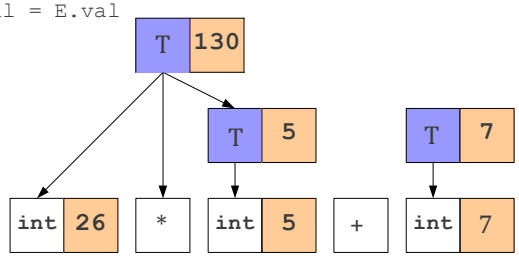
E → T + E	E_1.val = T.val + E_2.val
E → T	E.val = T.val
T → int	T.val = int.val
T → int * T	T.val = int.val * T.val
T → (E)	T.val = E.val



154

Simple Semantic Actions

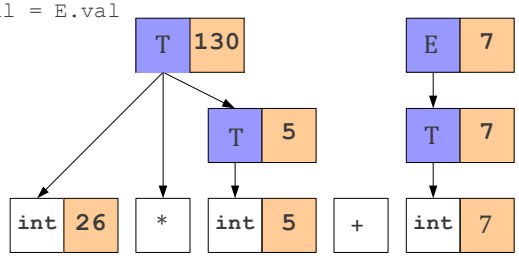
E → T + E	E_1.val = T.val + E_2.val
E → T	E.val = T.val
T → int	T.val = int.val
T → int * T	T.val = int.val * T.val
T → (E)	T.val = E.val



155

Simple Semantic Actions

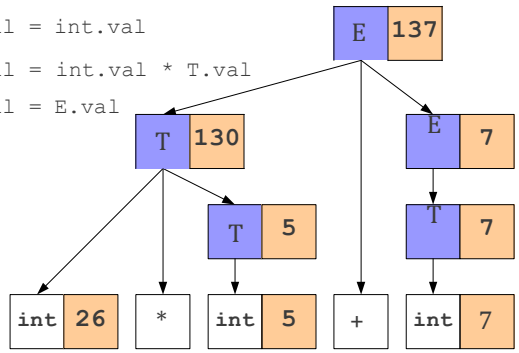
E → T + E	E_1.val = T.val + E_2.val
E → T	E.val = T.val
T → int	T.val = int.val
T → int * T	T.val = int.val * T.val
T → (E)	T.val = E.val



156

Simple Semantic Actions

E → T + E	E_1.val = T.val + E_2.val
E → T	E.val = T.val
T → int	T.val = int.val
T → int * T	T.val = int.val * T.val
T → (E)	T.val = E.val



Semantic Actions to Build ASTs

R → S	R.ast = S.ast;
R → R "|" S	R_1.ast = new Or(R_2.ast, S.ast);
S → T	S.ast = T.ast;
S → S T	S_1.ast = new Concat(S_2.ast, T.ast);
T → U	T.ast = U.ast;
T → T*	T_1.ast = new Star(T_2.ast);
U → a	U.ast = new SingleChar('a');
U → "ε"	U.ast = new Epsilon();
U → (R)	U.ast = R.ast;

Summary

- Syntax analysis (**parsing**) extracts the structure from the tokens produced by the scanner.
- Languages are usually specified by **context-free grammars** (**CFG**s).
- A **parse tree** shows how a string can be **derived** from a grammar.
- A grammar is **ambiguous** if it can derive the same string multiple ways.
- There is no algorithm for eliminating ambiguity; it must be done by hand.
- **Abstract syntax trees** (**AST**s) contain an abstract representation of a program's syntax.
- **Semantic actions** associated with productions can be used to build ASTs.